Evolução da Qualidade Estrutural de Projetos Spring: transição para Spring Boot e evidências de assistência por IA

Gabriel Ferreira[1], **Guilherme Augusto**[1]

[1]Engenharia de Software
Pontifícia Universidade Católica de Minas Gerais (PUC-MG)
Belo Horizonte – MG – Brazil

{gabriel.amaral.1447804,gajsouza}@sga.pucminas.br

Abstract. *This observational study follows 540 Spring candidate repositories from the original selection, created through 2017. The 20 oldest projects are examined in four eras and the remaining 520 in three, using the last mainline commit within each era. Of 1,640 planned project-era positions, 1,460 snapshots were recovered and 584 met the structural-comparison criteria. Five older projects could be compared between E1 and E4, and 123 later projects between E2 and E4. Coupling and class response increased in these paired sets, while other dimensions showed heterogeneous changes. In E4, 49 eligible projects with AI evidence had higher means across the seven measures than 157 without detected evidence, but also substantially larger codebases. The findings describe structural trajectories and observed associations, not causal effects of Spring Boot or AI, nor a single measure of community maturity.*

Resumo. *Este estudo observacional acompanha 540 candidatos Spring da seleção original, criados até 2017. Os 20 antigos são examinados em quatro eras e os 520 posteriores em três, usando o último commit da linha principal dentro de cada era. Das 1.640 posições projeto–era previstas, 1.460 snapshots foram recuperados e 584 atenderam aos critérios de comparação estrutural. Cinco projetos antigos puderam ser comparados entre E1 e E4, e 123 posteriores entre E2 e E4. Acoplamento e resposta das classes aumentaram nesses conjuntos pareados, enquanto outras dimensões apresentaram mudanças heterogêneas. Em E4, 49 projetos elegíveis com evidência de IA apresentaram médias maiores nas sete medidas que 157 sem evidência detectada, mas também tinham código substancialmente maior. Os resultados descrevem trajetórias e associações observadas, sem identificar efeitos causais de Spring Boot ou IA nem uma nota única de maturidade da comunidade.*

Bacharelado em Engenharia de *Software* - PUC Minas
Trabalho de Conclusão de Curso (TCC)

Orientador de conteúdo (TCC I): Danilo Maia - danilomaia@pucminas.br
Orientador de conteúdo (TCC I): João Pedro Batisteli - joaobatisteli@pucminas.br
Orientador de conteúdo (TCC I): Leonardo Vilela - leonardocardoso@pucminas.br
Orientador acadêmico (TCC I): Cleiton Tavares - cleitontavares@pucminas.br
Belo Horizonte, versão de trabalho de 15 de outubro de 2026.

1. Introdução

A qualidade interna de *software* descreve propriedades estruturais do código-fonte, como modularidade, coesão e acoplamento, que podem ser investigadas por análise estática [ISO/IEC 2011, Pressman and Maxim 2020]. Em sistemas orientados a objetos, a suíte de Chidamber e Kemerer oferece indicadores para caracterizar essas dimensões [Chidamber and Kemerer 1994]. Esses indicadores não constituem, isoladamente, uma nota de maturidade: sua interpretação depende do tamanho, do domínio e da organização do projeto.

O ecossistema Spring permite investigar a evolução de projetos Java ao longo de diferentes períodos tecnológicos. Projetos iniciados antes da consolidação do Spring Boot podem ter atravessado mudanças de configuração, dependências, práticas de desenvolvimento e ferramentas de assistência. O Spring Boot 1.0 estável foi lançado em 1ž de abril de 2014 [Spring 2014]; por isso, o período de 2009 a 2013 é tratado como contexto de Spring tradicional, com identificação explícita de eventuais versões preliminares de Boot. A presença atual de Boot em um repositório criado em 2010 não demonstra que essa tecnologia já integrava seu código em 2010.

A mesma distinção é necessária para a inteligência artificial. Um projeto antigo pode apresentar hoje vestígios de ferramentas de IA introduzidos recentemente. Datas de criação dos repositórios não datam o uso dessas ferramentas, e fronteiras de calendário não separam perfeitamente produção humana e assistida. O Copilot já estava em prévia técnica em junho de 2021 [GitHub 2021]. Evidências de arquivos, identidades e mensagens de commit permitem observar parte dessa adoção, mas não todo uso nem a proporção do código produzida por IA [Robbes et al. 2026].

A pergunta central deste trabalho é: **como a qualidade estrutural de projetos do ecossistema Spring evoluiu ao longo do tempo, e quais mudanças acompanharam a adoção do Spring Boot e o aparecimento de evidências de assistência por IA?** O foco está no acompanhamento dos mesmos projetos da seleção original, criados até 2017. Os 20 criados até 2013 permitem investigar versões anteriores à consolidação de Boot. A existência de candidatos antigos não garante que todos tenham versões tecnicamente elegíveis. Esse desenho distingue evolução dentro do projeto de diferenças de composição entre conjuntos de projetos.

O objetivo geral é caracterizar longitudinalmente a evolução estrutural dos projetos selecionados, recuperando a última versão disponível dentro de cada era do histórico Git. Os objetivos específicos são: (i) descrever trajetórias de acoplamento, complexidade, tamanho, herança e coesão; (ii) descrever mudanças nos mesmos projetos entre versões com diferentes configurações de Spring e Boot; (iii) explorar as mudanças que acompanham o aparecimento de evidências de IA; e (iv) situar essas trajetórias no contexto dos projetos disponíveis em cada época, por tamanho e framework. O paradigma GQM liga essas perguntas às sete medidas extraídas pela ferramenta CK. O acompanhamento histórico constitui o eixo principal; Boot e evidências de IA são características observáveis cuja análise depende da cobertura disponível. Os pontos de encerramento das eras não identificam necessariamente o instante exato de uma transição.

O interesse na evolução da comunidade é abordado por esse recorte de projetos, sem tratar a amostra como representação de todos os usuários de Spring. A idade de um

repositório, a adoção de uma ferramenta e a redução de uma métrica não são equivalentes a maturidade. Neste trabalho, qualidade estrutural designa as dimensões do código Java observadas pelos indicadores; não inclui uma avaliação direta de produtividade, funcionalidade, desempenho ou competência dos desenvolvedores.

A relevância do estudo está em produzir evidências históricas rastreáveis, com código, datas, versões e cobertura associados a cada medição. A mineração de repositórios possibilita esse acompanhamento, mas exige cuidado com seleção, disponibilidade do histórico e interpretação dos metadados [Hassan 2008, Kalliamvakou et al. 2016]. Mudanças de framework, linguagem, domínio, equipe e tamanho são explicações concorrentes para variações das métricas. Nem a injeção de dependências elimina a influência do framework sobre a estrutura, nem um contraste entre períodos permite atribuir diferenças exclusivamente ao desenvolvedor ou à IA.

As saídas incluem tabelas de cobertura, médias por época e diferenças entre versões do mesmo projeto, com registros de reprodução. Esta revisão apresenta a análise dos 540 candidatos originais, com 1.640 posições verificadas e 584 versões elegíveis. A investigação admite estabilidade, aumentos e reduções das métricas, sem presumir melhoria. A Seção 2 apresenta a fundamentação; a Seção 3, os trabalhos relacionados; a Seção 4, o protocolo histórico; a Seção 5, os resultados disponíveis; e a Seção 6, as conclusões e seus limites.

2. Fundamentação Teórica

Esta seção apresenta os conceitos que embasam o estudo: qualidade estrutural de *software*, métricas orientadas a objetos, o paradigma *Goal-Question-Metric*, o acompanhamento histórico por mineração de repositórios e as evidências de assistência por inteligência artificial.

2.1. Qualidade interna e métricas orientadas a objetos

A norma *ISO/IEC 25010* define qualidade de *software* como um conjunto de características que expressam o grau em que um sistema atende a necessidades explícitas e implícitas [ISO/IEC 2011]. Nesse modelo, a qualidade interna refere-se a propriedades do código-fonte observáveis por análise estática, incluindo modularidade, coesão e facilidade de manutenção. Pressman e Maxim destacam que sistemas com melhor qualidade interna tendem a exigir menor esforço de evolução e a permanecer viáveis por mais tempo [Pressman and Maxim 2020].

Para tornar essas características mensuráveis em código orientado a objetos, Chidamber e Kemerer propuseram um conjunto clássico de indicadores em nível de classe [Chidamber and Kemerer 1994]. Acoplamento entre objetos (*Coupling Between Objects* — CBO) descreve o grau de dependência de uma classe em relação a outras. Métodos ponderados por classe (*Weighted Methods per Class* — WMC) agrega a complexidade dos métodos. Profundidade da árvore de herança (*Depth of Inheritance Tree* — DIT) e número de filhos (*Number of Children* — NOC) caracterizam a hierarquia. Resposta de uma classe (*Response for a Class* — RFC) estima o conjunto de métodos potencialmente acionados a partir de uma mensagem. Falta de coesão em métodos (*Lack of Cohesion in Methods* — LCOM) indica o quanto os métodos de uma classe operam sobre conjuntos disjuntos de atributos. Essas métricas passaram a ser amplamente empregadas para caracterizar projeto, manutenção e falhas em sistemas orientados a objetos

[Singh et al. 2010, Radjenović et al. 2013, Aniche et al. 2016]. A forma operacional em que são extraídas neste trabalho, incluindo a ferramenta e as variantes adotadas, é especificada na Seção 4.

Neste estudo, WMC representa a complexidade estrutural agregada dos métodos, conforme a implementação do CK; não mede tempo de execução, consumo de recursos ou desempenho computacional. O CBO empregado é o indicador geral de dependências entre classes do CK, sem uma separação implementada entre acoplamento horizontal e vertical. Para coesão, utiliza-se a coluna `lcom*`, variante normalizada documentada pelo instrumento, e não a coluna `lcom` original. As seis medidas em nível de classe são complementadas por LOC por método; essa medida de tamanho não é apresentada como uma sétima integrante da suíte CK original.

Valores devem ser interpretados em conjunto e segundo a função do código. Uma redução de DIT ou NOC descreve uma mudança na hierarquia, sem estabelecer que o projeto se tornou melhor. Mesmo mudanças em complexidade ou acoplamento exigem contexto sobre responsabilidades e tamanho [Aniche et al. 2016]. Nenhuma métrica isolada prova violação de um princípio de projeto, nem identifica a autoria humana ou assistida do código.

2.2. Goal-Question-Metric

O paradigma *Goal-Question-Metric* (GQM) organiza avaliações de *software* em três níveis: o objetivo do estudo (*goal*), as perguntas que o detalham (*questions*) e as métricas que as respondem (*metrics*) [Basili et al. 1994]. A abordagem evita coletar indicadores sem vínculo com a intenção da pesquisa: primeiro define-se o que se deseja aprender; em seguida, traduz-se esse objetivo em perguntas mensuráveis; por fim, escolhem-se as medidas adequadas. Neste trabalho, o GQM liga o objetivo de analisar a evolução da qualidade estrutural em projetos do ecossistema Spring às dimensões de acoplamento, complexidade, herança, coesão e tamanho, operacionalizadas na Seção 4.

2.3. Mineração de Repositórios de Software

A Mineração de Repositórios de *Software* (*Mining Software Repositories* — MSR) analisa artefatos reais — código-fonte, histórico de versões e metadados de plataformas — para identificar padrões de desenvolvimento, evolução e qualidade [Hassan 2008]. Em vez de apoiar-se apenas em relatos ou documentação, a MSR observa evidências extraídas dos próprios repositórios. No GitHub, essa estratégia exige filtros de relevância, exclusão de *forks* e distinção entre criação na plataforma, idade do código e disponibilidade de versões, a fim de evitar a interpretação de metadados atuais como evidência histórica [Kalliamvakou et al. 2014, Kalliamvakou et al. 2016]. Esses cuidados conceituais orientam a amostragem descrita na metodologia.

2.4. Inteligência artificial generativa

Neste trabalho, inteligência artificial generativa designa sistemas baseados em modelos de linguagem de grande escala, treinados em corpora de texto e código, capazes de produzir ou completar trechos de programa a partir de contexto e de *prompts*. Ferramentas como GitHub Copilot, ChatGPT, Claude Code e Cursor ilustram formas distintas de assistência, introduzidas em momentos diferentes. E4, iniciada em 2022, é um recorte de calendário

do estudo; pertencer a essa época não confirma uso de IA, assim como estar em E3 não o exclui.

Experimentos sobre produtividade e qualidade de sugestões motivam investigar essas ferramentas, mas medem resultados diferentes das métricas estruturais de um repositório [Peng et al. 2023, Moradi Dakhel et al. 2023]. Também é necessário distinguir assistentes de completamento e agentes de codificação: estes podem deixar rastros mais explícitos, como coautoria em commits [Robbes et al. 2026]. Consequentemente, a distribuição temporal dos vestígios encontrados depende tanto da adoção quanto da possibilidade de detectá-la. O estudo acompanha evidências observáveis, sem equipará-las a todo uso de IA.

3. Trabalhos Relacionados

Os trabalhos relacionados são organizados em três frentes: acompanhamento histórico de sistemas, interpretação de métricas estruturais e evidências de assistência por IA. Essa organização situa a contribuição principal do TCC na evolução de projetos Spring, tratando a IA como uma possível mudança posterior em suas trajetórias.

3.1. Evolução de sistemas e mineração de repositórios

[Godfrey and Tu 2000] acompanharam 96 versões do núcleo Linux, investigando o crescimento do sistema e de seus subsistemas. O estudo encontrou trajetórias que exigiam examinar a organização interna, além dos valores agregados de tamanho. A relação com este TCC é metodológica: observar versões do mesmo sistema permite investigar mudanças que uma única medição atual não revela. O objeto e as medidas diferem, pois o presente trabalho acompanha vários projetos Java/Spring por snapshots ao final de cada era e métricas estruturais, sem transferir o padrão de crescimento do Linux para esse ecossistema.

A mineração de repositórios oferece meios para reconstruir essas trajetórias, mas a disponibilidade de dados não garante sua adequação à pergunta de pesquisa [Hassan 2008]. Os estudos de [Kalliamvakou et al. 2014, Kalliamvakou et al. 2016] discutem problemas de interpretação dos repositórios do GitHub e de sua relação com a atividade de desenvolvimento. Esse cuidado fundamenta a distinção, neste trabalho, entre um candidato encontrado por metadados, um histórico recuperável e uma versão tecnicamente elegível. A seleção original já excluiu arquivados e projetos sem atividade recente pelos metadados. Manter essa base assegura rastreabilidade da população autorizada, mas preserva o viés de sobrevivência e não recupera projetos desaparecidos.

3.2. Interpretação contextual das métricas

A abordagem SATT, de [Aniche et al. 2016], propõe limites de métricas específicos para papéis arquiteturais, em vez de avaliar todas as classes pelos mesmos valores de referência. O estudo examinou projetos MVC e Android e relacionou diferenças de métricas às responsabilidades das classes. Para este TCC, a implicação é que uma variação agregada pode refletir mudanças na composição arquitetural do projeto. O trabalho não aplica os limites SATT nem classifica automaticamente os projetos como bons ou ruins: utiliza as mesmas medidas ao longo do tempo e explicita o contexto de sua interpretação.

No contexto de injeção de dependências, [Sun et al. 2022] propõem medidas que distinguem formas de acoplamento relevantes à manutenibilidade. A investigação mostra por que métricas de dependência precisam ser interpretadas à luz dos mecanismos utilizados no código. Este TCC mantém o CBO geral do instrumento CK; portanto, não reproduz as medidas ponderadas daquele estudo nem assume que a injeção de dependências elimina o acoplamento observado. A comparação de variantes de CBO em [Child et al. 2019] também reforça a necessidade de documentar qual definição foi operacionalizada.

Esses trabalhos delimitam o alcance das conclusões: métricas estruturais descrevem propriedades do código, enquanto sua relação com qualidade depende do contexto. A consistência do extrator entre versões ajuda a comparar medições, mas não transforma uma redução numérica em evidência suficiente de melhoria da arquitetura.

3.3. Assistência por IA e identificação de vestígios

Estudos experimentais sobre Copilot e outros geradores analisam produtividade, correção e qualidade em tarefas delimitadas [Peng et al. 2023, Moradi Dakhel et al. 2023, Yetistiren et al. 2023]. O EvalPlus, apresentado por [Liu et al. 2023], amplia os testes usados para avaliar a correção funcional de soluções geradas. Esses resultados motivam a investigação da assistência por IA, mas não fornecem uma previsão direta para as métricas CK de sistemas completos. O presente estudo observa repositórios históricos; não reproduz os experimentos nem mede tempo de desenvolvimento ou correção funcional.

A abordagem BIMAN, de [Dey et al. 2020], combina características de nomes, mensagens e atividade para identificar bots que realizam commits. Essa identificação não equivale à detecção de assistência por IA: automações convencionais também produzem commits, e contribuições assistidas podem permanecer sob a identidade do desenvolvedor. Por isso, um autor automatizado precisa de interpretação contextual e a ausência de uma identidade de bot não caracteriza um projeto como exclusivamente humano.

[Robbes et al. 2026] investigam vestígios de agentes de codificação em 128.018 projetos do GitHub, distinguindo-os de assistentes de completamento. O estudo explora rastros como configuração e coautoria, que tornam parte dessa adoção observável. O detector deste TCC é uma adaptação local, com heurísticas próprias e inspeção temporal do histórico Git; sua precisão não é herdada daquele trabalho. Sinais de configuração, identidades e trailers são conservados como evidências distintas, sem determinar a parcela de código gerada ou a continuidade do uso.

A contribuição deste trabalho está na combinação entre versões históricas de projetos Spring, medições estruturais rastreáveis e marcos observáveis de framework e assistência por IA. A evolução dos projetos constitui o objeto principal. Os contrastes em torno de Boot e IA complementam essa descrição, sujeitos à quantidade e à qualidade das versões e evidências disponíveis.

4. Materiais e Métodos

Esta revisão, definida em 15 de outubro de 2026, restringe a investigação à seleção original do trabalho. O estudo é observacional, longitudinal e descritivo. A unidade acompanhada é o repositório, e a unidade medida é o último snapshot disponível dentro de cada

era. As comparações descrevem mudanças estruturais; não identificam efeitos causais de Spring Boot ou de assistência por IA.

4.1. Base de origem e filtragem anterior ao processamento

A entrada exclusiva é o arquivo `selected_repositories.csv` da execução `20260920_java`, com 38.456 identificadores únicos. Não são acrescentados repositórios de outras consultas ou anteriormente excluídos. A descoberta ampliada realizada em outubro pertence a um protocolo anterior e não constitui a população desta revisão.

Sobre os metadados salvos, aplica-se o limite de criação até o último dia de 2017 em UTC, operacionalizado por `created_at` anterior a `2018-01-01T00:00:00Z`. O recorte contém 540 candidatos: 20 criados até 2013 e 520 em 2014–2017. A filtragem exclui 37.916 registros e antecede a recuperação de históricos, a detecção de IA e a execução de CK. Arquivo de entrada, hashes, regras, candidatos e motivos de exclusão são preservados.

A composição confirmada inclui todos os 540 candidatos, sem sorteio ou substituição: os 20 antigos são acompanhados em E1, E2, E3 e E4; os 520 posteriores, em E2, E3 e E4. Estão previstas 1.640 posições projeto–era, distintas da quantidade de versões efetivamente recuperadas ou elegíveis. A lista e o corte temporal de E4 são congelados antes da coleta principal. E1 utiliza somente os 20 antigos originais.

4.2. Atividade e fundamentação dos limites

Preserva-se o critério de atividade já aplicado à seleção original: presença de `pushed_at` a partir de 17/09/2023 16:36:06 UTC, usando como referência a coleta de 17/09/2026. A data atual não desloca retrospectivamente esse corte. Oliveira et al. [Oliveira et al. 2021] utilizaram uma janela de três anos sem commits e mínimo de 1 KLOC em um estudo sobre especialistas em bibliotecas Java e microserviços. Trata-se de precedente metodológico com outro objetivo, não de validação universal para CK. Nosso uso de `pushed_at` é uma aproximação por metadados, distinta de verificar commits no histórico.

Uma comparação local com três meses, mantendo a mesma referência temporal, reteria 219 dos 540 candidatos. Essa análise de sensibilidade não altera a amostra. A restrição original à atividade pode introduzir viés de sobrevivência; as conclusões não abrangem automaticamente projetos antigos inativos ou desaparecidos.

4.3. Snapshots por era

As eras de medição são E1 (2009–2013), E2 (2014–2017), E3 (2018–2021) e E4 (2022 até o instante fixado da execução). E4 é parcial. A criação serve à seleção; a data do código determina a medição. Não se interpreta um projeto criado em 2010 e medido em 2026 como código de 2010.

Recupera-se todo o histórico alcançável da branch padrão, com referência e SHA fixados. Na linha do primeiro pai dos merges, escolhe-se o último commit cuja data de committer esteja dentro da era. Registram-se também autoria e maior data da ancestralidade. Datas posteriores ao corte geram diagnóstico. A data de committer é uma aproximação operacional de integração, não comprovação da publicação do código no GitHub.

Sem commit no intervalo, registra-se ausência de snapshot. Não se usa o HEAD atual, não se transporta um commit de outra era e não se troca a última versão por uma anterior mais conveniente. Em E1, verifica-se elegibilidade somente nos 20 projetos originais; falhas não são substituídas por outros repositórios. Migrações e reescritas de histórico continuam sendo limitações.

4.4. Contagem de tamanho e extração CK

O instrumento `analisar_painel_original.py` reutiliza o CK auditado e seu adaptador, sem compilar ou executar os projetos. Preservam-se médias por classe de CBO geral, RFC, WMC, DIT, NOC e LCOM*, além de LOC por método, os valores brutos e a separação entre produção e testes.

Mantêm-se os limites operacionais anteriormente documentados: 2.000 LOC e dez arquivos Java por snapshot. O limite de arquivos, ausente no extrator histórico, foi implementado. Não foi identificada nas referências consultadas validação do par exato de limites; eles permanecem escolhas explícitas do TCC. O estudo de Oliveira et al. utiliza 1 KLOC, não 2.000 LOC nem dez arquivos.

Cada caminho de arquivo Java é contado uma vez. As linhas físicas com código excluem linhas vazias e exclusivamente de comentário; classes internas não duplicam LOC. A soma de LOC das classes CK não aplica o corte. São registrados arquivos totais, arquivos com código e contagens de produção e testes. Essas definições explícitas seguem a preocupação de Park [Park 1992] com a comparabilidade das medidas de tamanho; a referência não fornece os limites mínimos de inclusão adotados aqui.

A comparação principal exige confirmação de Spring no conteúdo da versão, cobertura completa no diagnóstico adotado e sete métricas disponíveis. Código do próprio framework é separado de aplicações usuárias. Arquivos de build, código gerado e dependências seguem as exclusões documentadas no extrator. Falhas, ausência de histórico e métricas indisponíveis não são convertidas em zero. A resolução estática de dependências pode ser parcial. Configurações dinâmicas ou dependências que o detector não resolve podem produzir o diagnóstico de framework não confirmado. Esse diagnóstico não comprova ausência de Spring e não autoriza alterar a regra após observar CK. A presença de Boot significa declaração em pelo menos um módulo incluído no escopo; pode estar em exemplos ou módulos auxiliares. Portanto, não equivale à migração integral da arquitetura. Essa distinção também acompanha os casos de bibliotecas que acrescentam integração com Boot.

4.5. IA, comparações e reprodução

A detecção local inspeciona sinais em arquivos e commits disponíveis até cada snapshot, conservando separadamente presença na versão e evidência histórica. Sinais de branches laterais dependem de sua integração. Ausência de vestígio não significa ausência de uso; primeiro vestígio observado não é a data exata da adoção. Não se atribui à adaptação local a precisão do estudo que a inspirou.

Apresentam-se quantidades de candidatos, tentativas, versões elegíveis, médias aritméticas das sete medidas e diferenças pareadas dos mesmos projetos entre eras. Cada projeto contribui uma vez por era. Comparações com e sem evidência de IA são descritivas e têm denominadores explícitos. Não se apresentam p-valores ou deltas de Cliff.

Tabela 1. Questões e medidas do protocolo revisado (GQM).

Q	Questão	Medidas
Q1	Como a estrutura dos mesmos projetos varia entre eras?	Sete médias CK; diferenças pareadas; cobertura.
Q2	Quais mudanças acompanham configurações distintas de Spring e Boot?	Framework e versão declarada em cada SHA; métricas CK.
Q3	Quais diferenças aparecem entre versões e projetos com e sem evidências detectadas de IA?	Evidências temporalmente delimitadas; quantidades e médias CK.
Q4	Quanto a composição dos conjuntos e o tamanho variam entre eras?	IDs compartilhados, arquivos, LOC e motivos de exclusão.

Tamanho, arquitetura, domínio, equipe e seleção temporal são explicações concorrentes para mudanças observadas.

As médias gerais usam classes e métodos de produção e testes, com as duas categorias também preservadas separadamente nos dados. Médias entre projetos atribuem peso igual a cada repositório. Painéis balanceados incluem somente IDs elegíveis em todas as eras de sua coorte; comparações de duas eras exigem elegibilidade apenas nas duas versões envolvidas e informam o número de pares.

A validação cobre origem dos IDs, limites UTC, primeiro pai, ancestralidade futura, ausência de commits na era, tamanho, arquivos distintos e retomada. Execuções congeladas não são sobrescritas; alterações de instrumento exigem nova execução. O protocolo detalhado e os comandos estão em `METODOLOGIA_PAINEL_ORIGINAL.md`.

5. Resultados e Discussão

5.1. População e cobertura efetiva

A filtragem da seleção original reduziu 38.456 registros a 540 candidatos criados até 2017: 20 antigos e 520 criados em 2014–2017. Todos foram percorridos, sem substituição. Foram registradas as 1.640 posições previstas, com 1.460 snapshots recuperados. Houve 174 posições sem commit dentro da era e seis falhas de recuperação, correspondentes a dois repositórios indisponíveis em suas três eras previstas.

Entre os snapshots recuperados, 1.441 tiveram extração CK com estado completo no diagnóstico adotado; 15 não continham fontes Java no escopo, três apresentaram erro do extrator e um teve cobertura parcial. Extração completa não implica elegibilidade: confirmação do framework, tamanho e demais critérios reduziram a comparação principal a 584 versões, pertencentes a 245 projetos distintos. Os totais por era são versões de projetos que podem se repetir; não devem ser somados como repositórios únicos.

Tabela 2. Cobertura das posições projeto–era na seleção original.

Era	Previstas	Com SHA	Elegíveis	Sem commit
E1	20	20	6	0
E2	540	525	162	13
E3	540	472	210	66
E4	540	443	206	95

Tabela 3. Médias entre projetos elegíveis por era; a composição varia entre linhas.

Era	Projetos	CBO	RFC	WMC	LOC/método	DIT	NOC	LCOM*
E1	6	5,103	8,460	8,415	6,455	1,373	0,091	0,274
E2	162	5,773	8,285	7,416	5,847	1,291	0,084	0,276
E3	210	5,956	8,779	7,024	5,986	1,302	0,092	0,264
E4	206	6,150	9,663	7,017	6,225	1,297	0,088	0,257

Os 20 antigos tiveram suas quatro eras verificadas. Foram elegíveis seis versões em E1, nove em E2, onze em E3 e dez em E4. A versão de E1 do próprio Spring Boot foi identificada como código do framework; a presença preliminar de Boot no guia de serviço REST também foi registrada, mas seu tamanho não atingiu os limites principais. Assim, não se presumiu ausência de Boot antes do lançamento estável nem se confundiu código do framework com uma aplicação.

As falhas permanecem individualizadas nos dados. Dois erros ocorreram em versões de um projeto com arquivos de template e outro em um exemplo Java não processado pelo visitante do CK. A cobertura parcial envolveu caminhos Java que diferem apenas por maiúsculas/minúsculas. Nenhum desses casos foi convertido em métrica zero ou trocado por uma versão anterior conveniente.

5.2. Mudanças nos mesmos projetos

Cinco projetos antigos foram elegíveis em E1 e E4. Nesse conjunto, CBO e RFC aumentaram em média, enquanto WMC diminuiu. LOC por método, NOC e LCOM* variaram pouco em termos absolutos. Esses resultados descrevem dimensões distintas; não sustentam uma direção única de melhoria estrutural.

Nos 123 projetos criados em 2014–2017 elegíveis em E2 e E4, as médias de CBO, RFC, WMC e LOC por método aumentaram. DIT e NOC variaram pouco, e LCOM* diminuiu. A redução desse indicador de falta de coesão coexistiu com aumento de acoplamento e complexidade. Não se agrega esse comportamento em uma nota única de maturidade.

Tabela 4. Mesmos 5 projetos criados até 2013: E1 e E4; diferença = depois menos antes.

Métrica	Média E1	Média E4	Diferença
CBO	5,055	6,512	1,457
RFC	8,880	10,601	1,721
WMC	8,952	8,077	-0,875
LOC/método	6,684	6,695	0,012
DIT	1,428	1,401	-0,027
NOC	0,104	0,104	0,000
LCOM*	0,290	0,287	-0,002

Tabela 5. Mesmos 123 projetos criados em 2014–2017: E2 e E4; diferença = depois menos antes.

Métrica	Média E2	Média E4	Diferença
CBO	5,836	6,131	0,295
RFC	8,289	9,812	1,523
WMC	6,898	7,126	0,228
LOC/método	5,928	6,264	0,336
DIT	1,296	1,293	-0,003
NOC	0,089	0,087	-0,002
LCOM*	0,269	0,259	-0,010

Os gráficos do relatório mantêm composição constante em todas as eras de cada coorte: quatro projetos antigos em E1–E4 e 118 posteriores em E2–E4. Esses conjuntos são menores que os pares das tabelas, pois exigem elegibilidade também nas eras intermediárias. A cobertura pode alterar a leitura: nos quatro antigos do painel balanceado, LCOM* passou de 0,279 para 0,302, enquanto nos cinco pares E1–E4 houve pequena redução de 0,290 para 0,287. Portanto, explicitar os IDs compartilhados é necessário mesmo em uma análise descritiva.

5.3. Presença de Spring Boot e evidências de IA

Entre endpoints adjacentes elegíveis, a passagem observável de Spring tradicional para presença declarada de Boot ocorreu em quatro pares E1–E2, nove E2–E3 e quatro E3–E4. A diferença média de CBO nesses conjuntos foi, respectivamente, $-0,260$, $+1,136$ e $-0,498$. Os conjuntos e períodos distintos não apresentaram uma direção uniforme. A presença de Boot em um módulo ou exemplo não equivale à migração integral do projeto; esses números não isolam um efeito do framework.

Em E4, 94 candidatos apresentaram evidência detectada de IA, 349 não apresentaram os sinais procurados e 97 permaneceram indeterminados: 95 sem snapshot na era e dois sem recuperação do repositório. Entre os 206 snapshots elegíveis para CK, 49 tinham evidência e 157 não tinham evidência detectada. Não foram encontrados sinais nas versões de E1–E3 pelo detector adotado; isso não comprova inexistência de assistência nesses períodos.

Tabela 6. E4: médias entre projetos elegíveis com evidência de IA (A, $n_a = 49$) e sem evidência detectada (B, $n_b = 157$).

Métrica	Média A	Média B	A menos B
CBO	6,580	6,016	0,564
RFC	11,767	9,007	2,760
WMC	8,142	6,667	1,475
LOC/método	6,468	6,149	0,319
DIT	1,334	1,286	0,048
NOC	0,098	0,084	0,014
LCOM*	0,276	0,251	0,025

As sete médias de E4 foram maiores no grupo com evidência. Entretanto, esse grupo também apresentou maior porte: média de 125.578 LOC totais por projeto, contra

15.222 no grupo sem evidência detectada. A comparação não controla tamanho, domínio, arquitetura ou equipe e não permite concluir que a IA aumentou complexidade ou piorou qualidade.

Nos 43 pares elegíveis com ausência de vestígio em E3 e evidência observada em E4, CBO médio passou de 6,356 para 6,618. Nos 139 pares sem vestígio nos dois endpoints, passou de 5,925 para 5,975. A diferença entre esses conjuntos já aparecia na versão de E3. Além disso, a primeira evidência observada não data o primeiro uso real. A análise representa associação temporal, sem identificação causal.

5.4. Alcance das evidências

O resultado central é uma descrição longitudinal rastreável, com cobertura explícita e mudanças heterogêneas entre dimensões. A seleção original por tópicos atuais, atividade e arquivamento preserva viés de sobrevivência. Configurações não resolvidas pelo detector, perdas de histórico, falhas do CK e assistência sem vestígios também restringem o alcance. Os poucos projetos antigos comparáveis impedem generalizar sua trajetória para toda a comunidade Spring. Os resultados de populações anteriores não foram incorporados a essas tabelas.

6. Conclusão

O estudo acompanhou os 540 candidatos da seleção original criados até 2017, verificando 1.640 posições projeto–era e obtendo 584 versões elegíveis para a comparação estrutural. Cinco projetos antigos puderam ser comparados entre E1 e E4, e 123 criados em 2014–2017 entre E2 e E4. A disponibilidade de um repositório antigo não garantiu cobertura elegível em todas as eras.

As trajetórias revelaram mudanças heterogêneas: aumentos de acoplamento e resposta das classes coexistiram com reduções ou estabilidade de outras medidas. A direção também dependeu do conjunto de versões comparáveis. Consequentemente, os dados não sustentam uma afirmação única de que a comunidade melhorou ou piorou sua maturidade estrutural.

Em E4, 49 projetos elegíveis com evidência de IA apresentaram médias maiores nas sete medidas que os 157 sem evidência detectada, mas também eram, em média, maiores em LOC. As diferenças anteriores ao aparecimento observável dos sinais e as limitações do detector impedem atribuir essas diferenças à IA. Da mesma forma, as mudanças que acompanharam presença declarada de Boot não identificam um efeito causal do framework.

A contribuição é o painel reproduzível que distingue criação, data do código, disponibilidade, elegibilidade e evidência observada. A continuidade do estudo pode ampliar a validação independente dos sinais, examinar papéis arquiteturais e avaliar sensibilidade aos limites de tamanho, sempre preservando a origem dos candidatos e declarando alterações de protocolo.

Referências

Aniche, M., Treude, C., Zaidman, A., van Deursen, A., and Gerosa, M. A. (2016). SATT: Tailoring code metric thresholds for different software architectures. In *2016 IEEE*

16th International Working Conference on Source Code Analysis and Manipulation (SCAM), pages 41–50. IEEE.

Basili, V. R., Caldiera, G., and Rombach, H. D. (1994). The goal question metric approach. In *Encyclopedia of Software Engineering*, pages 528–532. John Wiley & Sons, New York.

Chidamber, S. R. and Kemerer, C. F. (1994). A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493.

Child, M., Rosner, P., and Counsell, S. (2019). A comparison and evaluation of variants in the coupling between objects metric. *Journal of Systems and Software*, 151:120–132.

Dey, T., Mousavi, S., Ponce, E., Godfrey, T., Padhy, B., Nguyen, A., and Mockus, A. (2020). Detecting and characterizing bots that commit code. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR 2020)*, pages 209–219, Seoul, Republic of Korea. ACM.

GitHub (2021). Introducing github copilot: your ai pair programmer. `https://github.blog/news-insights/product-news/introducing-github-copilot-ai-pair-programmer/`. Publicado em 29 de junho de 2021.

Godfrey, M. W. and Tu, Q. (2000). Evolution in open source software: A case study. In *Proceedings of the International Conference on Software Maintenance*. IEEE.

Hassan, A. E. (2008). The road ahead for Mining Software Repositories. In *2008 Frontiers of Software Maintenance*, pages 48–57. IEEE.

ISO/IEC (2011). Systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — system and software quality models. International Standard ISO/IEC 25010:2011, International Organization for Standardization, Geneva.

Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., and Damian, D. (2014). The promises and perils of mining GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR '14)*, pages 92–101, Hyderabad, India. ACM.

Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., and Damian, D. (2016). An in-depth study of the promises and perils of mining GitHub. *Empirical Software Engineering*, 21(5):2035–2071.

Liu, J., Xia, C. S., Wang, Y., and Zhang, L. (2023). Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. Curran Associates.

Moradi Dakhel, A., Majdinasab, V., Nikanjam, A., Khomh, F., Desmarais, M. C., and Jiang, Z. M. (2023). GitHub Copilot AI pair programmer: asset or liability? *Journal of Systems and Software*, 203:111734.

Oliveira, J., Viggiato, M., Pinheiro, D., and Figueiredo, E. (2021). Mining experts from source code analysis: An empirical evaluation. *Journal of Software Engineering Research and Development*, 9.

Park, R. E. (1992). Software size measurement: A framework for counting source statements. Technical Report CMU/SEI-92-TR-020, Software Engineering Institute, Carnegie Mellon University.

Peng, S., Kalliamvakou, E., Cihon, P., and Demirer, M. (2023). The impact of AI on developer productivity: evidence from GitHub Copilot. arXiv:2302.06590.

Pressman, R. S. and Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education, New York, 9 edition.

Radjenović, D., Heričko, M., Torkar, R., and Živković, A. (2013). Software fault prediction metrics: a systematic literature review. *Information and Software Technology*, 55(8):1397–1418.

Robbes, R., Matricon, T., Degueule, T., Hora, A., and Zacchiroli, S. (2026). Agentic much? adoption of coding agents on GitHub. arXiv:2601.18341v2. Versão de 8 de abril de 2026. Consultado em 11 de outubro de 2026.

Singh, Y., Kaur, A., and Malhotra, R. (2010). Empirical validation of object-oriented metrics for predicting fault proneness models. *Software Quality Journal*, 18(1):3–35.

Spring (2014). Spring boot 1.0 ga released. `https://spring.io/blog/2014/04/01/spring-boot-1-0-ga-released/`. Publicado em 1 de abril de 2014.

Sun, P., Kim, D.-K., Hua, M., and Lu, L. (2022). Measuring impact of dependency injection on software maintainability. *Computers*, 11(9):141.

Yetistiren, B., Ozsoy, I., Ayerdem, M., and Tuzun, E. (2023). Evaluating the code quality of AI-assisted code generation tools: an empirical study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT. arXiv:2304.10778.

Apêndice A — Dados e reprodução do painel original

Os artefatos desta revisão estão na execução `20261015_base_original_ate2017`, sob `Instrumentos/Codigos/Artefatos/evolucao_spring/runs/`. O arquivo `sample.csv` contém os 540 identificadores congelados; `e1_20.csv` e `e2_520.csv` explicitam os dois recortes de criação. Não há sorteio nesta composição. A semente registrada no formato geral do manifesto não altera a inclusão de todos os candidatos.

`measurements.csv` registra cada posição projeto–era, com SHA, datas, framework, classificação de IA, arquivos, LOC, sete medidas, elegibilidade e motivos de exclusão. A trajetória dos 20 antigos também está em `e1_trajectories.csv`. Os arquivos brutos de classes, métodos e inventário de fontes permanecem comprimidos por chave de extração. Contagens de produção e testes são separadas; o corte usa LOC físicos por arquivo, não a soma dos tamanhos das classes.

`paired_summary.csv` compara os mesmos IDs elegíveis nas duas eras de cada par. `balanced_panel.csv` informa os IDs compartilhados em todas as eras da coorte. `ai_ck_comparison.csv` explicita as médias e os denominadores dos grupos: n_a corresponde a projetos com evidência e n_b a projetos sem evidência detectada, ambos elegíveis para CK. Esses denominadores não são contagens de classes, métodos ou commits.

Os manifestos de filtragem, amostra e instrumento conservam hashes e limites temporais. O auditor verifica identidade dos registros, completude das posições previstas, escolha de commits, ancestralidade, arquivos efetivamente presentes no SHA, inventário de LOC, médias contra os dados brutos e delimitação temporal das evidências de IA. Falhas de extração podem coexistir com auditoria aprovada quando estão corretamente registradas como falhas e excluídas da síntese CK; a auditoria não as transforma em medições válidas.

Os instrumentos são `analisar_painel_original.py`, `auditar_painel_original.py` e `relatorio_painel_original.py`. Os comandos e dependências estão em `METODOLOGIA_PAINEL_ORIGINAL.md`. Os relatórios anteriores, incluindo o protocolo ampliado e seus apêndices, permanecem arquivados no repositório; seus resultados não integram as tabelas desta revisão.